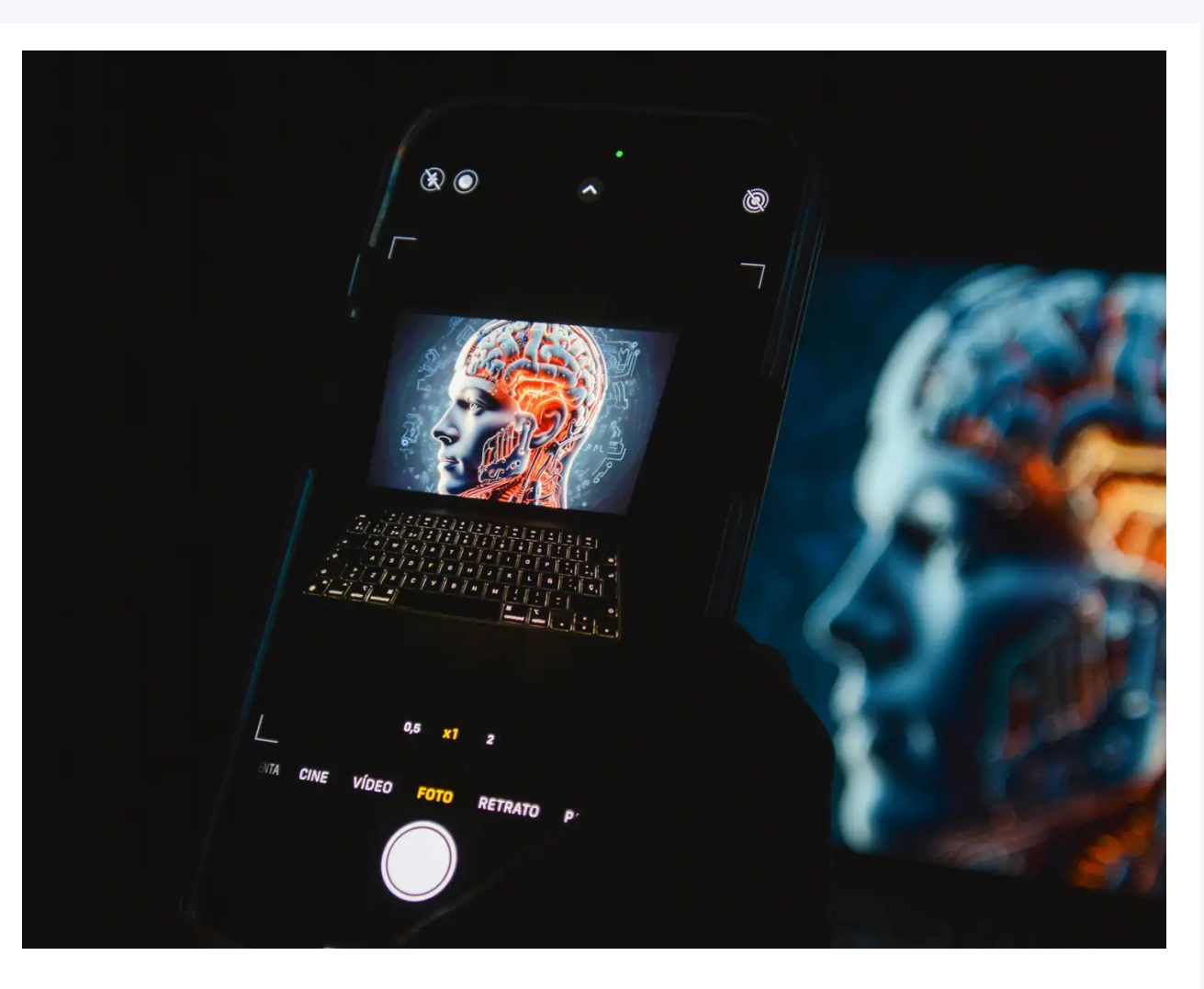

大模型Prompt技巧全解析

腾讯技术工程
编程话题下的优秀答主

已关注

收录于 · 大模型应用 ›

83 人赞同了该文章 ›

作者：mitin

在数字化浪潮的推动下，AI大模型以其卓越的自然语言处理能力和智能交互特性，迅速在很多领域中占据了重要地位。比如：与传统客服相比，AI大模型展现出了无可比拟的优势，通过精心设计的 prompt，能够让我们在客服托管、智能客服等多个项目和业务场景中发挥显著作用，大幅提高工作效率，优化成果质量。为了精进个人能力以及助力产研学习氛围的提升，本人在学习完吴恩达教授以及其他前辈们有关 prompt 的课程之后，整理了这份学习资料，以供大家学习交流。这篇笔记将深入探讨 prompt 的定义和撰写框架，详细阐述打造高效 prompt 的原则与策略，同时从技术原理层面剖析 prompt，助力读者更好地运用大模型，提升业务价值。

第一节：Prompt定义及基本框架

一、什么是Prompt

大模型 prompt 指在大语言模型应用中，用于"提示"模型唤起特定能力以解决实际问题的提问方式。早期它被称作"输入形式"或"输入模板"，后来"prompt"（提示）这一叫法因契合大语言模型语境，能准确体现其在调用模型能力方面的关键作用，成为公认术语。

大模型多数核心能力在预训练阶段形成，prompt 就像钥匙，引导模型从预训练积累的海量信息中，精准唤起如理解复杂文本、总结信息、生成内容、逻辑推理等能力，满足用户实际需求。它并非把模型当作单纯知识库，而是高效调用其能力，实现类似人类运用智能解决复杂问题的效果，是 prompt 工程的核心，对用好大语言模型意义重大。

二、 撰写Prompt的基本框架

1、Prompt包含的要素

- 指令：想要模型执行的特定任务或指令。
- 上下文：包含外部信息或额外的上下文信息，引导语言模型更好地响应。
- 输入数据：用户输入的内容或问题。
- 输出指示：指定输出的类型或格式。

2、Prompt五大框架

2.1 RTF框架

RTF(Role-Task-Format)框架是一个非常简单通用的Prompt提示框架，我们和任意大模型对话场景下都可以使用该规范进行改进输出。

- R-Role(角色)：指定大模型担当固定角色(程序员、数据分析师、讲解员、记者等等)
- T-Task(任务): 任务，告诉大模型需要为我们做的事情
- F-Format(格式)：大模型最终结果的返回格式(比如：表格、Markdown、英文等等)

主要优点：

- 简单、方便。
- 指定Role角色，可以让大模型在当前的角色范围内回答知识，这在一些特定的领域中非常有效。
- 指定Role角色也能让工程上检索知识能够确定边界范围，配合元数据所发挥的威力会更强。
- 如果结合RAG知识内容检索，那么上下文回答的内容会让用户感觉更加是顺畅。

2.2 思考链模式

通过这种模式来逐步改善大模型的推理能力，非常适合一些复杂的任务处理。例如：

- 分析型或者逻辑推理型的任务
- 决策
- 解决问题(比如程序员根据错误日志找Bug)

而要使用这种模式，只需要在末尾添加"让我们逐步思考"即可。例如：

```
# 数据源（与指令区分）
user_datasource = """XXX……"""

prompt1 = """分析一下在人工客服服务场景下，'''{user_datasource}'''中客户有哪些诉求
prompt2 = """分析一下在人工客服服务场景下，'''{user_datasource}'''中客户有哪些诉求

# 模型输出结果
output1：在人工客服服务场景下，客户主要诉求为：微信账号存在安全风险导致无法添加好友、单点
output2：在人工客服服务场景下，客户主要诉求为：微信账号存在安全风险导致无法添加好友，以及

# 分析结果
output1中的"单点拦截"并不是用户的诉求，而output2显然更加正确
```

2.3 RISEN框架

- R-Role:大模型扮演的角色
- I-Instructions: 指示命令，和Task-任务差不多
- S-Steps: 步骤
- E-End Goal: 最终目标
- N-Narrowing(Constraints): 缩小范围(约束条件)，和RTF框架中的Format有异曲同工之妙，一个是格式的约束，而这里的约束可以是任意方面，比如回答的内容(特定领域)、字数限制等等方面

该框架主要适合：

- 撰写具有特定约束的任务(例如博客文章)
- 有明确指导方针的任务（例如商业计划）

2.4 RODES框架

- R-Role: 角色
- O - Objective: 目标
- D - Details: 详细的细节
- E - Examples: 示例
- S - Sense Check: 感官检查

2.5 密度链模式

密度链模式Prompt是Salesforce、麻省理工学院和哥伦比亚大学的研究人员推出的一种新提示，它非常的高效，使用递归来创建越来越好的输出的提示，与普通提示生成的 GPT-4 摘要相比，它生成的摘要更加密集且更适合人们理解。

适合：

- 总结
- 改进您最喜欢的提示
- 通过递归生成可用的长格式内容

(感兴趣的同学可以自己检索一下！)

第二节：打造高效 Prompt的两大核心原则

在 Prompt 工程领域，掌握精妙的撰写策略是释放语言模型强大潜力的关键。本节将深入探讨prompt撰写的两大核心策略，为您揭示如何通过优化指令，引导模型生成更贴合需求、更具质量的输出。

一、原则一：编写明确和具体的指令

为了让模型精准理解我们的意图，编写清晰、明确且具体的指令至关重要。这不仅能避免模型产生误解，还能显著提升输出结果的相关性和准确性。

策略1：使用分隔符清晰界定输入部分

在构建prompt时，使用分隔符将特定文本部分与提示的其他部分清晰隔开，能有效避免提示词冲突。任何能让模型识别出单独部分的符号都可作为分隔符，常见的包括：

- 章节标题：通过不同层级的标题区分不同内容模块。
- 三重双引号："""，在代码编写中常用于包裹较长的文本字符串。
- 三重单引号：'''，功能与三重双引号类似，适用于不同的语法环境。
- 三重破折号：---，在文本中起到明显的分隔作用。
- 角括号：<>，常被用于标记特定元素。
- XML 标签：，利用结构化的标签形式区分不同内容。

示例1：假设我们需要对一段产品描述进行总结，可通过如下方式使用分隔符：

```
product_description = f"""这是一款全新的智能手表，具备心率监测、睡眠追踪等健康功能，
prompt =  f"""请总结以下产品描述：'''{product_description}'''"""
```

示例2：如果要让模型对一段用户评论进行情感分析，示例如下：

```
user_comment = f"""这款手机的拍照效果真的太棒了，色彩还原度高，夜景模式也很出色，唯一
prompt = f"""分析以下用户评论的情感倾向：''' {user_comment}''' """
```

这种方式能够确保模型明确区分指令和待处理文本，避免因指令混淆而产生错误输出。

策略2：要求结构化输出

为了便于后续对模型输出进行解析和处理，我们可以要求模型以特定的结构化格式输出，如HTML或JSON。这种方式能极大提高数据处理的效率，尤其是在Python编程环境中，结构化输出可直接读入字典或列表中，方便进一步的分析与操作。

示例1：要求模型以 JSON 格式输出。

```
prompt = "请以json格式列出每个nodeType所体现的用户诉求、客服方案。每一个json-value

# 数据源（与指令区分）
user_datasource = """XXX……"""

# 模型输出
{
"IVR": {
"用户诉求": "微信使用问题求助",
"客服方案": "引导正常使用或自助处理"
},
"ASYNC": {
"用户诉求": "账号封禁等相关申诉",
"客服方案": "按规则处理或引导自助"
},
"AI": {
"用户诉求": "支付相关问题咨询",
"客服方案": "要求用户详细描述问题"
}
}
```

示例2：以HTML格式输出一个简单的书籍。

```
prompt = "请以HTML格式列出三本你推荐的科幻小说及其作者。"
# 模型输出
<ul>
    <li>《三体》 - 刘慈欣</li>
    <li>《基地》 - 艾萨克·阿西莫夫</li>
    <li>《银河帝国：机器人五部曲》 - 艾萨克·阿西莫夫</li>
</ul>
```

策略3：要求模型检查条件是否满足

在面对复杂任务时，如果在一些假设条件，而这些条件并非总是成立，那么我们需要引导模型首先对这些假设进行检查。若条件不满足，模型应明确指示并停止继续执行任务，以避免无效计算和错误结果。同时，考虑潜在的边缘情况，并为模型设定相应的处理规则，能够有效减少意外错误，确保模型输出的稳定性和可靠性。

示例：假设我们要求模型计算一个数学表达式，但前提是表达式所有都为正数。

```
expression = "5 + 3 - (-2)"
prompt = f"""首先检查表达式 '{expression}' 中的所有数字是否都为正数。如果是，请计算
# 模型输出
表达式中存在非正数，无法计算
```

策略4：Few-shot prompting（少样本提示）

在要求模型执，提供成功完成任务的示例，能够帮助模型更好地理解任务要求和期望输出的格式。通过这些示例，模型可以学习到任务的模式和规律，从而在处理实际输入时生成更符合预期的结果。少样本提示尤其适用于处理一些复杂或具有特定格式要求的任务，能够显著提升模型的表现。

示例：要求模型按照特定格式将中文词汇翻译成英文并造句。

```
prompt = """
示例1：
中文词汇：苹果
翻译：apple
造句：I like to eat apples.

示例2：
中文词汇：学校
翻译：school
造句：I go to school every day.

中文词汇：图书馆
翻译：
造句：
"""
# 模型输出
翻译：library
造句：I often read books in the library.
```

二、原则二：给予模型充足的思考时间

当模型因急于得出结论而出现推理错误时，我们需要通过优化式，为模型提供更清晰的推理路径，引导其进行更深入、全面的思考。

策略1：明确完成任务所需的步骤

为模型详细指定完成任务所需遵循的步骤，能够帮助模型有条不推理和计算。通过这种方式，模型可以更清晰地理解任务的逻辑结构，避免因思路混乱而产生错误。例如，要求模型按照特定格式输出，能够引导其在处理过程中注重信息的组织和呈现方式。

示例：要求模型分析一首古诗，格式为"诗歌主题 - 主要意象 - 表达情感"。

```
poem = "床前明月光，疑是地上霜。举头望明月，低头思故乡。"
prompt = f"""请按照"诗歌主题 - 主要意象 - 表达情感"的格式分析这首诗：{poem}"""
# 模型输出
思乡之情 - 明月 - 对故乡的深切思念
```

策略2：引导模型在得出结论前充分思考方案

明确指示模型在得出最终结论前，先进行充分的推理和分析，往往能获得更优的结果。这种方式能够促使模型深入挖掘问题的本质，综合考虑各种因素，从而避免因片面或仓促的判断而导致的错误。

示例：问模型"如何提高公司的员工满意度"。

```
prompt = "在回答如何提高公司的员工满意度之前，请先思考至少三个方面的因素，然后综合这些因
# 模型输出
首先，从薪酬福利方面，可适当提高薪资水平、完善福利体系，如增加带薪休假天数等；其次，从职业
```

三、模型的局限性与应对策略

尽管语言模型在大量数据的训练下展现出强大的能力，但它们并非完美无缺。其中，"幻觉"现象是模型面个重要挑战。

模型"幻觉"

即使模型在训练过程中接触了海量知识，但它并不能完美记住所有信息，也难以精准把握自身知识的边界。这就导致在面对晦涩主题的问题时，模型可能会编造出看似合理但实际上错误的内容，这就是所谓的"幻觉"。

解决幻觉的一种策略

为了有效应对幻觉问题，我们可以要求模型在回答问题前，首先从给定文本中寻找相关引用，并依据这些引用来构建答这种方式，模型的回答将基于可靠的信息来源，而非凭空臆测，从而大大提高回答的准确性和可信度。同时，这种方法还能够实现答案的可追溯性，便于对模型输出进行验证和审查。

示例：给定一段关于历史事件的文本，要求模型回答相关问题。

```
text = """1861年，美国爆发了南北战争，这场战争是美国历史上的重要转折点，它主要围绕奴隶
prompt = f"""请从上述文本中找出相关信息，回答南北战争主要围绕什么问题展开？"""
# 模型输出
南北战争主要围绕奴隶制的存废问题展开，北方主张废除奴隶制，南方则坚持保留。
```

第三节：Prompt技术剖析与应用

在深入探索 prompt 技术的征程中，我们将逐一解析多种关键技术，助力您精准把握其核心要点与应用场景，切实提升应用能力。

一、零样本提示（Zero-Shot Prompting）

零样本提示是一种让模型在没有特定任务示例展示的情况下直接处理任务的技术。其原理在于模型经过大量数据训练和指令调整后，自身具备了一定的通用知识和任务理解能力。例如：

```
prompt = "判断此文本的情感倾向为积极、消极或中性：这部电影情节紧凑，特效惊人。情感："
# 模型会依据自身预训练知识对这个提示进行处理并输出结果
```

模型能依据自身预训练知识输出"积极"。然而，由于缺乏具体示例引导，对于复杂任务，其效果可能受限。

二、少样本提示（Few-Shot Prompting）

少样本提示在零样本提示效果不佳时发挥作用。它通过为模型提供少量任务示例，帮助模型学习任务模式和规律。例如：

```
prompt = """"whatpu"是坦桑尼亚的一种小型毛茸茸的动物。一个使用 whatpu 这个词的句子的
# 模型会根据前面的示例学习并生成新的句子
```

通过提供如上述新词汇造句的示例，模型能够模仿示例的结构和逻辑生成新的内容。不过在复杂推理任务中，仍需进一步优化。

三、思维链提示（Chain-of-Thought Prompting）

思维链提示旨在为模型提供清晰的推理步骤引导，从而显著提升其在复杂推理任务中的表现。它通过在提示中展示详细的推理过程，让模型学习如何逐步分析和解决问题。比如：

```
prompt = """Roger 有 5 个网球，他又买了 2 罐网球，每罐有 3 个球，他现在共有多少球？
Roger 一开始有 5 个球。
2 罐网球，每罐 3 个，共 6 个球。
5 + 6 = 11。
答案是 11。"""
# 模型按照提示中的推理步骤进行计算和回答
```

此技术可与少样本提示结合，增强效果，尤其适用于算术、常识推理等任务，帮助模型更有条理地处理问题。

四、自我一致性（Self-Consistency）

自我一致性技术主要用于优化思维链提示中的推理路径选择。其核心思想是通过提供多个少样本推理示例，让模型从多样的推理结果中筛选出最一致的答案，增强模型在算术和常识推理任务中的可靠性。比如：

```
prompt = """Q：林中有 15 棵树。林业工人今天将在林中种树。完成后，将有 21 棵树。林业工
A：我们从 15 棵树开始。后来我们有 21 棵树。差异必须是他们种树的数量。因此，他们必须种了
Q：停车场有 3 辆汽车，又来了 2 辆汽车，停车场有多少辆汽车？
A：停车场已经有 3 辆汽车。又来了 2 辆。现在有 3 + 2 = 5 辆汽车。答案是 5。
Q：当我 6 岁时，我的妹妹是我的一半年龄。现在我 70 岁了，我的妹妹多大？
A："""
# 模型会参考前面的推理示例对最后一个问题进行回答，并从多个可能的回答中选择最一致的答案
```

五、生成知识提示（Generated Knowledge Prompting）

生成知识提示主要用于解决模型在处理需要额外知识的任务时出现的局限性。它的操作方式是先让模型生成与任务相关的知识，再整合这些知识得出准确答案。比如：

```
prompt = """输入：高尔夫球的一部分是试图获得比其他人更高的得分。
知识：高尔夫球的目标是以最少的杆数打完一组洞。一轮高尔夫球比赛通常包括 18 个洞。每个洞在
解释和答案："""
# 模型根据生成的知识对问题进行回答
```

像这样先生成相关知识，再整合知识得出准确答案"不是，高尔夫球的目标是以最少杆数完成比赛，而非追求更高得分"，有效提升模型在常识推理任务中的准确性。

六、链式提示（Prompt Chaining）

链式提示是将复杂任务拆解为多个子任务，通过逐个子任务生成提示并传递结果的方式来实现复杂任务的有序处理。以文档问答为例：

```
# 第一个提示用于提取相关引文
prompt1 = """你是一个很有帮助的助手。你的任务是根据文档回答问题。第一步是从文档中提取与
# 假设这里有一个文档变量 doc，将其代入提示中进行处理
# 处理后得到引文列表，假设为 quotes_list

# 第二个提示根据引文和文档生成回答
prompt2 = """根据从文档中提取的相关引文（由<quotes></quotes>分隔）和原始文档（由###
# 模型根据第二个提示生成最终回答
```

这种方式提高了任务处理的准确性与可靠性，使得模型能够逐步深入地处理复杂问题。

七、思维树（ToT）

思维树框架是为了帮助模型应对复杂的探索性任务而设计的。它通过维护一棵思维树，让模型在解决问题时能够生成和评估中间思维步骤，并结合搜索算法进行系统性探索。示例如下：

```
# 假设给定数字为 4、9、10、13
prompt = """输入: 4 9 10 13
4 + 9 = 13 (left 10 13 13)
10 - 4 = 6 (left 6 9 13)
13 - 6 = 7 13 - 9 = 4
4 + 6 = 10 (left 7 9 4 * 6 = 24 (left 4 6)
评估是否能得到 24: sure
"""
# 模型按照思维树的步骤进行推理和计算，并根据评估结果继续搜索或得出答案
```

八、检索增强生成（RAG）

检索增强生成（RAG）技术将信息检索与文本生成相结合，专门用于处理知识密集型任务。它通过检索相关文档来为模型提供额外的知识支持，从而缓解模型的"幻觉"问题。例如：

```
# 假设这里有一个检索函数 retrieve_documents 和一个生成答案的函数 generate_answer
question = "谁是首次登上月球的人？"
documents = retrieve_documents(question)
prompt = "根据以下文档回答问题：" + str(documents) + " 问题是：" + question
answer = generate_answer(prompt)
# 最终得到的 answer 即为模型结合检索结果生成的答案
```

这有效确保答案基于可靠知识源，在自然问题、事实验证等基准测试中表现卓越。

九、自动推理并使用工具（ART）

自动推理并使用工具（ART）技术使模型能够自动生成包含推理步骤的程序，并在需要时调用外部工具。例如：

```
prompt = """计算 16 个苹果平均分给 4 个人，每人几个，再加上 3 是多少？
请生成解决这个问题的程序步骤。"""
# 模型会生成类似下面的程序步骤
# result = (16 / 4) + 3
# 然后可以在代码中执行这个程序步骤得到最终答案
```

在 BigBench 和 MMLU 基准测试中，ART 在多种任务上表现突出，显著超越少样本提示和自动 CoT 等方法，提升了模型解决问题的能力和效率。

十、自动提示工程师（APE）

自动提示工程师（APE）技术能够自动生成和筛选任务指令。它通过利用大型语言模型生成指令候选项，再依据评估分数选择最佳指令，从而提升提示生成的效率与效果。比如：

```
# 假设这里有一个生成指令候选项的函数 generate_candidates 和一个选择最佳指令的函数 se
article = "这里是一篇新闻文章的内容"
candidates = generate_candidates(article)
prompt = select_best_instruction(candidates) + " " + article
# 模型根据最终的 prompt 生成文章总结
```

例如总结新闻文章时，它先利用大型语言模型生成如"提取文章关键人物、事件和时间""概括文章主要内容并突出重点"等指令候选项，再依据评估分数选择最佳指令，指导模型生成高质量总结。

十一、Active-Prompt

Active-Prompt 技术主要用于解决思维链示例有效性的问题。它通过先查询模型生成多个答案，计算不确定度后挑选最不确定的问题由人类注释示例，再用新示例推断其他问题，从而优化模型对不同任务的适应性。比如：

```
# 假设这里有一个查询模型生成答案的函数 query_model 和一个计算不确定度的函数 calculate
prompt = "对以下文本进行分类：这是一篇科技类文章。"
answers = query_model(prompt, num_answers=5)  # 生成 5 个答案
uncertainty = calculate_uncertainty(answers)
if uncertainty > threshold:  # 如果不确定度大于设定阈值
    # 选择最不确定的问题由人类注释示例，然后更新提示并重新查询模型
    new_prompt = update_prompt_with_annotated_example(prompt)
    final_answer = query_model(new_prompt)
else:
    final_answer = select_most_consistent_answer(answers)
# 最终得到的 final_answer 即为经过 Active-Prompt 处理后的答案
```

通过这种方式，能够提高模型在文本分类等任务中的准确性。

十二、方向性刺激提示（Directional Stimulus Prompting）

方向性刺激提示通过训练策略 LM 生成引导提示，增强对模型生成结果的掌控力。例如文本摘要任务：

```
article = "这里是一篇文章的内容"
prompt = "请根据文章内容生成一个引导提示，重点关注文章的主要事件和结果。"
# 策略 LM 会根据这个提示生成如"重点提取文章中事件的主体、发生的时间和最终结果"的引导提示
# 然后可以将这个引导提示用于后续的文本摘要任务中
```

在文本摘要任务中，策略 LM 依据文章主题、关键信息等生成引导提示，指导模型生成符合需求的摘要，提升摘要质量。

十三、PAL（程序辅助语言模型）

PAL（程序辅助语言模型）技术让模型生成程序来解决问题，借助编程运行时提升解决复杂问题的能力。以日期计算问题为例：

```
question = "今天是 2023 年 2 月 27 日，我出生于 25 年前，我的出生日期是多少？"
prompt = """
import datetime
from dateutil.relativedelta import relativedelta

# Q: 2015 年在 36 小时后到来。从今天起一周后的日期是多少（格式为 MM/DD/YYYY）？
# 如果 2015 年在 36 小时后到来，那么今天是 36 小时前。
today = datetime(2015, 1, 1) - relativedelta(hours=36)
# 从今天起一周后，
one_week_from_today = today + relativedelta(weeks=1)
# 格式化后的答案
one_week_from_today.strftime('%m/%d/%Y')

# Q: 今天是 2023 年 2 月 27 日，我出生于 25 年前，我的出生日期是多少？
"""
# 模型会生成类似下面的代码
# today = datetime(2023, 2 27)
# born = today - relativedelta(years=25)
# born.strftime('%m/%d/%Y')
# 然后可以在代码中执行模型生成的代码得到最终答案
```

十四、ReAct 框架

ReAct 框架使模型交错生成推理轨迹和操作，提升答案的可靠性与可解释性。以回答问题为例：

```
question = "除苹果遥控器外，哪些设备可控制其设计交互的程序？"
prompt = """问题：除苹果遥控器外，哪些设备可控制其设计交互的程序？
思考 1：我需要搜索苹果遥控器相关程序及可替代控制设备。
操作 1：搜索 [苹果遥控器 可替代控制设备]
观察 1：搜索结果为……
思考 2：根据搜索结果，分析哪些设备符合要求。
操作 2：筛选 [符合要求的设备]
观察 2：筛选结果为……
思考 3：整理最终答案。
操作 3：整理 [最终答案]
```

```
"""
# 模型会按照提示中的推理轨迹和操作步骤进行处理，并最终得出答案
```

在知识密集型和决策任务中表现良好，通过与外部环境交互获取信息来辅助推理。

十五、自我反思（Reflexion）

自我反思框架包含参与者、评估者和自我反思三个模型，旨在帮助模型从错误中学习并提升性能。模拟编程任务的示例如下：

```
# 假设这里有参与者模型 actor、评估者模型 evaluator 和自我反思模型 reflexion
code = "这里是一段初始代码"
# 参与者模型生成尝试解决问题的代码和动作
result = actor(code)
# 评估者模型评价输出
score = evaluator(result)
# 自我反思模型根据评价结果生成改进建议
reflection = reflexion(score, result)
# 将改进建议应用到下一次的代码生成中，实现自我学习和提升
```

如代码出现语法错误，自我反思模型会提示可能错误原因及修正方向，帮助模型快速学习。

附录

一、术语表

1. **Prompt**：在大语言模型应用中，用于“提示”模型唤起特定能力以解决实际问题的提问方式，是prompt工程的核心。
2. **RTF框架**：Role - Task - Format框架，通过指定角色、任务和格式规范大模型输出，简单通用。
3. **思考链模式**：在提示末尾添加“让我们逐步思考”，引导大模型逐步分析复杂任务，改善推理能力。
4. **RISEN框架**：包含角色、指示命令、步骤、最终目标和缩小范围（约束条件），适合特定约束或明确指导方针的任务。
5. **RODES框架**：涵盖角色、目标、详细细节、示例和感官检查的提示框架。
6. **密度链模式**：利用递归创建更好输出的提示，生成的摘要更密集且适合理解，适用于总结等任务。
7. **少样本提示（Few - shot prompting）**：为模型提供少量任务示例，帮助其学习任务模式和规律，提升在复杂或特定格式任务中的表现。
8. **零样本提示（Zero - Shot Prompting）**：模型在无特定任务示例展示下，依靠自身预训练知识和任务理解能力直接处理任务。
9. **思维链提示（Chain - of - Thought Prompting）**：为模型展示详细推理过程，提升其在复杂推理任务中的表现，可与少样本提示结合。
10. **自我一致性（Self - Consistency）**：通过提供多个少样本推理示例，让模型筛选最一致答案，增强在算术和常识推理任务中的可靠性。
11. **生成知识提示（Generated Knowledge Prompting）**：先让模型生成与任务相关知识，再整合知识得出准确答案，解决模型处理需额外知识任务的局限性。
12. **链式提示（Prompt Chaining）**：将复杂任务拆分为多个子任务，逐个子任务生成提示并传递结果，提高任务处理准确性与可靠性。
13. **思维树（ToT）**：通过维护思维树，让模型生成和评估中间思维步骤，结合搜索算法应对复杂探索性任务。
14. **检索增强生成（RAG）**：将信息检索与文本生成结合，检索相关文档为模型提供知识支持，缓解“幻觉”问题。
15. **自动推理并使用工具（ART）**：使模型自动生成包含推理步骤的程序，并在需要时调用外部工具，提升解决问题能力和效率。
16. **自动提示工程师（APE）**：利用大型语言模型生成指令候选项，依据评估分数选择最佳指令，提升提示生成效率与效果。
17. **Active - Prompt**：先查询模型生成多个答案，计算不确定度后挑选最不确定问题由人类注释示例，优化模型对不同任务的适应性。
18. **方向性刺激提示（Directional Stimulus Prompting）**：训练策略LM生成引导提示，增强对模型生成结果的掌控力，提升文本摘要等任务的质量。
19. **PAL（程序辅助语言模型）**：让模型生成程序来解决问题，借助编程运行时提升解决复杂问题的能力。
20. **ReAct框架**：使模型交错生成推理轨迹和操作，提升答案的可靠性与可解释性，在知识密集型和决策任务中表现良好。
21. **自我反思（Reflexion）**：包含参与者、评估者和自我反思三个模型，帮助模型从错误中学习并提升性能。

二、常见问题解答

1.如何选择合适的Prompt框架?

根据任务类型和需求选择。如简单对话场景可选用RTF框架；复杂分析任务可考虑思考链模式或RISEN框架；对输出格式有严格要求的，可在RTF、RISEN框架中关注格式或约束条件设定；进行总结类任务，密度链模式可能更合适。

2.为什么模型会出现“幻觉”现象?

模型虽经大量数据训练，但无法完美记住所有知识，也难以精准把握知识边界，面对复杂或生疏主题问题时，就可能编造看似合理但错误的内容。

3.如何利用少样本提示提升模型性能?

提供与任务相关、格式规范的成功示例，示例要涵盖任务的关键要素和模式，让模型学习示例中的规律和逻辑，从而在处理实际输入时生成更符合预期的结果。尤其适用于复杂任务或对输出格式有特定要求的情况。

4.链式提示在实际应用中有哪些优势?

将复杂任务拆解为子任务，使模型处理过程更清晰、有序，逐步深入解决问题，提高了任务处理的准确性与可靠性，避免模型在处理复杂问题时因任务过于复杂而出现混乱或错误。

发布于 2025-02-17 18:16 · 广东

大模型 LLM（大型语言模型） Prompt工程